AWE Chat Application

Project 1: Analyze the Scope of the Task(s) and User Requirements

Student Name: Eddie Chongtham
Course: SODV2452-01 – Application Development
Institution: Bow Valley College
Date: July 12, 2026
File: chongtham_eddie_assessment_part_1

1. Introduction

This document analyzes the scope and user requirements for the **AWE Chat** application — a Windows Forms desktop chat client written in C#. Users enter their name and exchange messages with others through a central server managed by a separate backend team. This document covers the client-side application and its integration point with the server via TCP.

The application has two Windows Forms: a **Login Form** and a **Chat Form**. Analysis is based on the AWE Chat Requirements Document (Sammy) and the provided starter code.

2. Codebase Assessment

2.1 Starting Points — What Exists

The starter code contains two projects:

File	Purpose	Location
`Program.cs`	Client application entry point. Contains the `sendMessage(byte[])` method — the integration interface provided by the backend team for sending/receiving messages over TCP. This is the only client-side code that exists; no UI has been built yet.	WinFormsApp1/

File	Purpose	Location
`TCPServer.cs`	Server simulator for local testing. Listens on `127.0.0.1:1234`, reads an incoming message, decodes it with `Encoding.Unicode`, and echoes it back. Provided by the backend team — not to be modified.	Server/
`WinFormsApp1.sln`	Visual Studio solution file tying both projects together.	Root

2.2 Missing Elements — What Needs to Be Built

Missing Element	Description
Login Form (UI)	No Windows Form exists yet. A Login Form with a Name field, Remember Me checkbox, and Login button must be created.
Chat Form (UI)	No Chat Form exists. A Chat Form with a Message input, History (read-only), and Send button must be created.
Input Validation	No validation logic exists. Must prevent empty Name from opening the Chat window and show an error message.
Data Persistence Layer	No storage mechanism exists. Must save/load the username when Remember Me is checked (SQL Server, flat file, or config file).
Error Handling in UI	The current `catch` block only prints to console — no user-facing error feedback exists. Must surface connection failures to the UI.
Async Network Calls	`sendMessage` is synchronous. Must be made async (`async/await`) to prevent blocking the UI thread.

2.3 Where Data Gets Processed

Step	What Happens	Location
1. Encode	The client encodes a text message into a `byte[]` using `Encoding.Unicode` (UTF-16) before sending.	Client — caller of `sendMessage`
2. Send	`sendMessage` creates a `TcpClient`, opens a `NetworkStream`, and writes the bytes to the server via `stream.Write()`.	`Program.cs`
3. Server Processes	The server reads the bytes, calls `cleanMessage()` to strip null characters, then re-encodes and	`TCPServer.cs`

Step	What Happens	Location
	writes the message back to the client stream.	
4. Receive	`sendMessage` reads the server's response bytes via `stream.Read()` into a 1 MB buffer and returns the byte array to the caller.	`Program.cs`
5. Display	The caller decodes the response bytes and appends the message to the History field on the Chat Form. *(Not yet implemented — to be built.)*	Chat Form (to be built)

3. Project Scope

3.1 In Scope and Out of Scope

> **No Gold Plating:** Only features explicitly required by the client (Sammy) or directly necessary to support them are in scope. All other enhancements are excluded.

Status	Item	Reason
IN SCOPE	Login Form with Name field, Remember Me checkbox, Login button	Explicitly required by Requirements Document
IN SCOPE	Name validation (empty check + error message)	Explicitly required by Requirements Document
IN SCOPE	Chat Form with Message field, History field (read-only), Send button	Explicitly required by Requirements Document
IN SCOPE	TCP integration using provided `sendMessage(byte[])` interface	Required to communicate with backend server
IN SCOPE	Data persistence for username (SQL Server / flat file / config file)	Required to support Remember Me feature
IN SCOPE	Connection error feedback to user	Required for reliability and usability
OUT OF SCOPE	Server-side development	Server is managed by the backend team; not this team's responsibility
OUT OF SCOPE	User authentication / passwords	Not mentioned in requirements — no gold plating

Status	Item	Reason
OUT OF SCOPE	Chat rooms or channels	Not mentioned in requirements — single shared chat only
OUT OF SCOPE	Message encryption (TLS/SSL)	Not required by stakeholder — beyond stated scope
OUT OF SCOPE	File sharing or media messages	Not mentioned in requirements
OUT OF SCOPE	Mobile or web client	Platform is explicitly Windows desktop (WinForms)

3.2 Limitations

Limitation	Explanation
No persistent chat history across sessions	The History field is cleared when the application closes, **because** the requirements only specify displaying messages during the current session — no session log storage is required.
Single server endpoint (hardcoded)	The server address is fixed at `127.0.0.1:1234`, **because** the backend team has defined this as the integration contract for the local simulator and no requirement for configurable endpoints was stated.
Only username is persisted	The persistence layer stores only the user's name, **because** the Requirements Document specifies no other data to be saved — storing more would be gold plating.
No real-time push (polling required)	The provided `sendMessage` interface is request-response only, **because** the server simulator does not implement a push/subscription model — continuous message receipt would require polling or a new server interface.
Windows only	The application runs on Windows exclusively, **because** Windows Forms is a Windows-only UI framework as specified in the requirements.

3.3 Feasibility Assessment

Requirement	Feasibility	Verdict
Login Form + Chat Form (WinForms)	Standard WinForms development — well-supported in C# .NET. Visual Studio designer makes this straightforward.	**Feasible**

Requirement	Feasibility	Verdict
TCP integration via `sendMessage`	Interface already provided by backend team. Client only needs to call it with encoded bytes. No custom networking logic needed.	**Feasible**
Remember Me (data persistence)	Storing a single string to a config file or SQLite/SQL Server is a solved problem in .NET. Local SQL Server may require installation on client machines.	**Feasible** (flat file or config file recommended to reduce setup)
Non-blocking UI (async/await)	C# async/await is built into .NET. Converting `sendMessage` to async is a well-understood pattern.	**Feasible**
Real-time multi-user chat	The current server simulator is echo-only (returns the sender's message). True multi-user broadcasting requires server-side changes. This is outside client scope.	**Partially feasible** (client-side limited by server capability)

3.4 Requirements Priorities

Priority	Requirement	Justification
P1 — Must Have	Login Form + Chat Form UI	Core deliverable — without these forms the application does not exist.
P1 — Must Have	Name validation	Explicitly required; prevents incorrect application state.
P1 — Must Have	Send message via TCP	Core chat functionality — the primary purpose of the application.
P1 — Must Have	Display messages in History	Core chat functionality — users cannot participate without seeing messages.
P2 — Should Have	Remember Me / data persistence	Explicitly required by client; improves UX for returning users.
P2 — Should Have	Connection error feedback	Required for reliability; prevents silent failures from confusing users.
P3 — Nice to Have	Async I/O + retry logic	Improves robustness and responsiveness but application functions without it.

4. Functional Requirements

What the system must *do*, extracted from the AWE Chat Requirements Document.

ID	Requirement	Source
FR-01	**Login Window:** App displays a Login window on startup with a Name field, Remember Me checkbox, and Login button.	Requirements Doc
FR-02	**Name Validation:** Login button validates name is not empty. If empty → show error message; do not open Chat window.	Requirements Doc
FR-03	**Open Chat:** If name is valid, close Login window and open Chat window.	Requirements Doc
FR-04	**Remember Me:** If checkbox is checked → save name to storage. On next launch → pre-populate Name field with saved name.	Requirements Doc
FR-05	**Message Field:** Chat window has a text input for composing messages.	Requirements Doc
FR-06	**Send Button:** Clicking Send encodes the message and calls `sendMessage(byte[])` to transmit to the server.	Requirements Doc + Program.cs
FR-07	**History Field:** Non-editable field displaying all sent and received messages in the current session, in chronological order.	Requirements Doc
FR-08	**TCP Communication:** App communicates with the server using the provided `sendMessage(byte[])` interface over TCP at `127.0.0.1:1234`.	Program.cs
FR-09	**Data Storage:** Username persistence uses one of: local SQL Server (recommended), flat file, or configuration file.	Requirements Doc
FR-10	**Error Feedback:** If a connection error occurs, display a visible error message to the user. Do not crash.	Requirements Doc + Program.cs catch block

5. Non-Functional Requirements

Quality attributes and constraints the system must satisfy.

ID	Requirement	Category
NFR-01	**Platform:** Runs on Windows as a WinForms .NET desktop application.	Platform
NFR-02	**Language:** Developed in C#.	Technology
NFR-03	**Encoding:** Messages encoded in Unicode (UTF-16) to match the server's `Encoding.Unicode`.	Interoperability

ID	Requirement	Category
NFR-04	**Usability:** A new user can log in and send a first message within 30 seconds of launch.	Usability
NFR-05	**Responsiveness:** UI stays responsive at all times — TCP operations must not block the UI thread.	Performance
NFR-06	**Reliability:** Application does not crash on connection failure; user sees a meaningful error message.	Reliability
NFR-07	**Validation:** Empty Name field triggers an error message and prevents the Chat window from opening.	Correctness
NFR-08	**Data Security:** Stored username is local-only; not transmitted beyond the chat protocol.	Security
NFR-09	**Maintainability:** Separation of concerns — UI, network, and persistence layers are distinct.	Maintainability
NFR-10	**Resource Management:** TCP connections and streams are disposed after every transaction.	Reliability

6. User Stories

Format: *"As a [role], I want to [action] so that [benefit]."*

ID	User Story	Acceptance Criteria
US-01	As a **new user**, I want to enter my name and click Login so that the app knows who I am before I chat.	• Login window shown on launch with Name field and Login button. • Non-empty name → Chat window opens. • Empty name → error shown, Chat window stays closed.
US-02	As a **returning user**, I want to check Remember Me so that my name is saved and pre-filled next time I open the app.	• Remember Me checkbox visible on Login form. • Checked + Login → name saved to storage. • Next launch → Name field pre-populated. • Unchecked → nothing saved, field blank next launch.
US-03	As a **user**, I want to type a message and click Send so that I can communicate with others in the chat.	• Message field and Send button on Chat window. • Send transmits the message to the server.

ID	User Story	Acceptance Criteria
		• Message field clears after sending.
US-04	As a **user**, I want all messages shown in the History field so that I can follow the conversation.	• History field is non-editable. • All sent and received messages appear in order with sender's name. • Auto-scrolls to most recent message.
US-05	As a **user**, I want to be told when a message fails to send so that I know it was not delivered.	• Visible error message shown on connection failure. • App does not crash. • Failed message not silently discarded.
US-06	As a **developer**, I want a defined `sendMessage` interface so that I can integrate with the server without knowing its internals.	• `sendMessage(byte[])` accepts Unicode-encoded bytes and returns the server's response bytes. • Connection failures surface a meaningful error to the caller.

7. Software Resilience Strategy

7.1 Exception Handling Design

The current `sendMessage` in **Program.cs** has a broad `catch (Exception e)` that only prints to the console — no error reaches the UI and the method returns a zeroed byte array with no signal of failure. The following design replaces this with structured exception handling:

Exception Type	Trigger	Handling Action
`SocketException`	Server not running, port refused, or read/write timeout exceeded.	Catch specifically; show dialog: *"Could not connect to the chat server. Please try again."* Log the error code.
`IOException`	Network stream drops mid-transfer (cable pull, Wi-Fi drop).	Catch specifically; notify user the message may not have been delivered; attempt reconnect.
`TimeoutException`	Server accepts connection but does not respond within the	Catch; show dialog: *"The server is not responding. Check your connection and try again."*

Exception Type	Trigger	Handling Action
	timeout window (5 seconds).	
`ObjectDisposedException`	Stream or client used after disposal (e.g., in a race condition).	Catch; log internally; do not surface technical details to user. Show generic retry prompt.
General `Exception`	Any unexpected error not covered above.	Catch as fallback; log full stack trace to a local error log file; show: *"An unexpected error occurred."*

7.2 When Things Go Wrong — Scenario Documentation

Scenario	What Breaks	Current Behaviour	Required Behaviour
Server is not running	`TcpClient` constructor throws `SocketException` (connection refused).	Exception message printed to console only. Method returns empty byte array. UI receives no signal — appears to send successfully.	UI displays: *"Cannot connect to the server. Please ensure the server is running."* Send button re-enabled.
Server crashes mid-message	`stream.Read()` returns 0 or throws `IOException`.	Exception swallowed. Caller gets a zeroed 1 MB buffer, attempts to decode garbage, may display corrupted text.	Detect zero-byte read; treat as disconnection; notify user message was not confirmed; log the event.
Server is slow / hangs	`stream.Read()` blocks indefinitely — no timeout configured.	UI thread freezes completely if called synchronously. Application appears to crash from	Set `ReceiveTimeout = 5000 ms`. Use `async/await`. If timeout: show *"Server not responding."*

Scenario	What Breaks	Current Behaviour	Required Behaviour
		user perspective.	
Large message / fragmented response	Single `stream.Read()` call returns fewer bytes than the full message.	Partial message silently returned. History field shows truncated or garbled text.	Loop on `stream.ReadAsync()` until full response received. Use message-length header for framing.
Repeated failures exhaust ports	`TcpClient` and `NetworkStream` not disposed when exception thrown before `client.Close()`.	Each failed send leaks a TCP connection. After many failures the OS runs out of available ports; application stops working.	Wrap in `using` blocks. Resources disposed whether success or failure.
User clicks Send rapidly	Multiple concurrent TCP connections opened before earlier ones complete.	Race condition — multiple threads read/write concurrently; responses may be delivered out of order.	Disable Send button while a send is in progress. Re-enable on completion or failure.

7.3 Resilience Properties Validation

Property	How to Validate	Pass Condition
Fail gracefully	Start the client without the server running. Click Send.	Error dialog shown. App does not crash. Send button re-enabled.
No UI freeze	Introduce a 10-second delay in the server. Click Send. Try to type in the Message field.	UI remains responsive. Message field accepts input. Timeout fires after 5 seconds.

Property	How to Validate	Pass Condition
No resource leak	Send 100 messages rapidly with and without server running. Check Windows Task Manager for handle count.	Handle count does not grow unboundedly.
Complete message delivery	Send a message >10,000 characters. Verify History field shows the complete message.	No truncation. All characters appear correctly in History.
Correct error type propagation	Simulate each failure scenario in section 7.2. Verify the correct dialog appears for each.	User sees specific, helpful error message matching each failure type.

7.4 Data Backup Strategy

To prevent data loss, the following backup considerations apply:

- **Username persistence file:** The flat file or config file storing the username should be written atomically (write to a temp file, then rename) so a crash during write does not corrupt the saved name.
- **Unsent messages:** If a send operation fails, the message should be preserved in memory (outgoing queue) so the user can retry rather than losing it.
- **Chat history:** Session chat history is in-memory only by design (out of scope per requirements). No backup needed for the current scope.

7.5 Implementation Priority for Resilience Fixes

1. **Must fix first:** Resource disposal with `using` + read/write timeouts — prevents port exhaustion and freezes.
2. **Must fix second:** Convert to `async/await` + propagate exceptions to UI — keeps app responsive and gives user feedback.
3. **Should add:** Message framing (length-prefix) + disable Send button during send — correctness and UX.
4. **Nice to have:** Retry with exponential backoff + outgoing message queue — production-quality reliability.